UNIVERSIDAD PANAMERICANA

Facultad de Ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y TIC

Análisis y Diseño de Sistemas

Hipstagram

(P3. Diseño de Arquitectura)

Jefry Fernando Berduo Pérez (000147971)

María Alejandra Tesagüic González (000151403)

Chimaltenango, marzo 2026

Hipstagram

(P3. Diseño de Arquitectura)

Jefry Fernando Berduo Pérez (000147971)

María Alejandra Tesagüic González (000151403)

Quinto Semestre, Ing. Mario Obed Morales Guitz (Docente)

Chimaltenango, marzo 2026

Índice

1 Introducción 5

2 Definiciones, acrónimos y abreviaturas 6

3 Diagrama de componentes 8

4 Descripción de componentes 9

5 Vista lógica de la arquitectura de los componentes de software 10

6 Diagrama de capas 11

7 Descripción de la seguridad y credenciales 12

7.1 Autenticación y Gestión de Sesiones (JWT) 12

7.2 Protección de credenciales 12

7.3 Control de Acceso Basado en Roles (RBAC) 12

7.4 Seguridad de la comunicación: 12

8 Diagrama de seguridad 13

9 Vista de Implementación 14

10 Descripción general de la implementación 15

11 Diagrama de la implementación 16

12 Vista de Puesta en Marcha 17

13 Vista de Datos 18

13.1 Capa de Dominio (Entidades de Negocio) 18

13.2 Capa de Servicios 19

13.3 Capa de Seguridad 19

14 Diagrama Entidad-Relación (DER) 20

15 Diccionario de Datos 21

16 Definición de Roles de la Aplicación 24

17 Alcance y Limitaciones 24

17.1 Alcance 24

17.2 Limitaciones 25

1 Introducción

Las plataformas para intercambio visual modernas como Hipstagram demandan una alta velocidad y manejo de grandes volúmenes de datos, por lo que utilizar una estructura monolítica tradicional no es recomendable debido a que todos sus componentes están relacionados entre sí y esto dificulta la ampliación y mantenimiento de una aplicación.

En respuesta a esta problemática, el diseño arquitectónico de Hipstagram está fundamentado en una arquitectura de microservicios. Esta estructura permite desglosar las funcionalidades tales como la gestión de usuarios, publicaciones y el análisis de votos, en unidades autónomas e independientes. Este enfoque garantiza una mayor resiliencia y agilidad, permitiendo que cada servicio evolucione sin afectar la estabilidad de la plataforma.

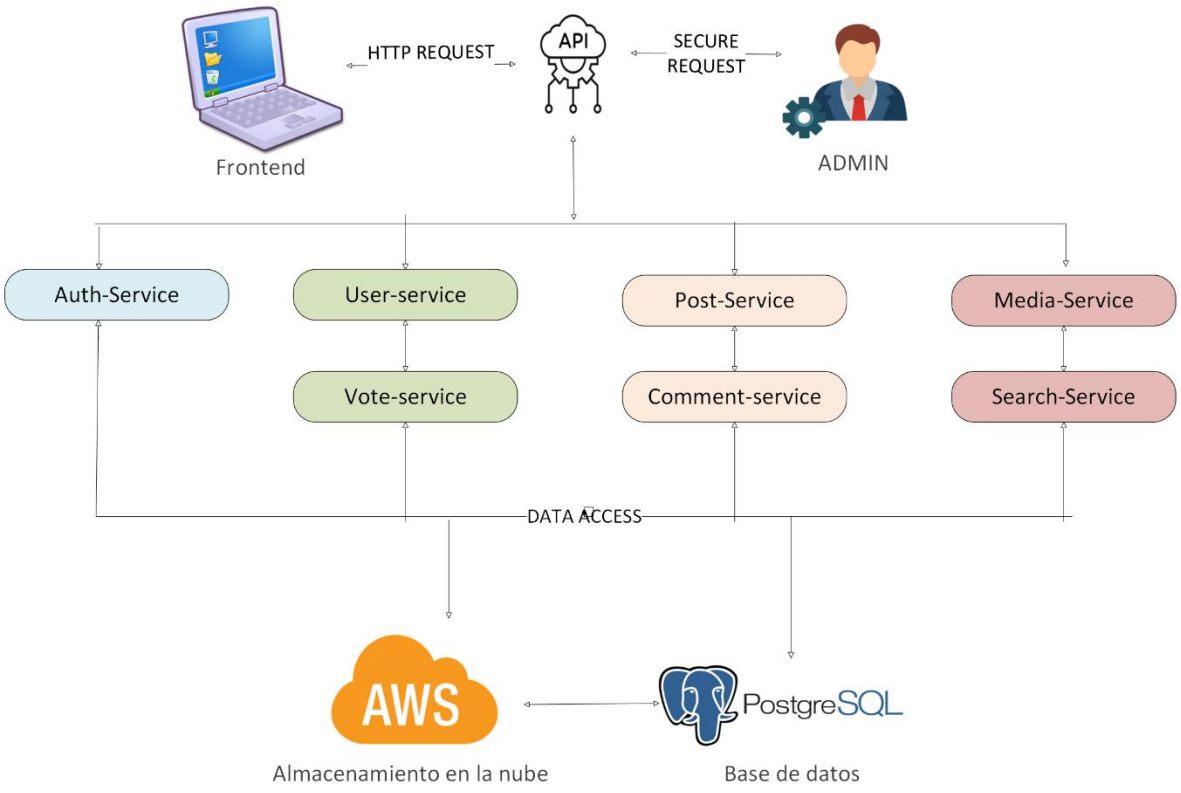
Para la implementación, se ha seleccionado un stack tecnológico de alto rendimiento: PostgreSQL como sistema de gestión de bases de datos relacionales por su robustez e integridad de datos, y Angular basado en TypeScript bajo un modelo de PWA (Progressive Web App) para ofrecer una experiencia multiplataforma consistente. El ecosistema se apoya en la infraestructura de AWS, utilizando Docker para la contenedorización y despliegue, y asegurando el acceso mediante el estándar de industria JWT (JSON Web Token). Esta combinación técnica asegura que Hipstagram sea una solución segura, escalable y de alta disponibilidad.

2 Definiciones, acrónimos y abreviaturas

- Access Token: Clave temporal de corta duración que autoriza cada petición del usuario a los microservicios.
- Admin (Administrador): Rol con privilegios elevados para moderación y gestión de la plataforma.
- API (Application Programming Interface): Conjunto de reglas y especificaciones que permiten que el Frontend y el Backend se comuniquen entre sí.
- AWS (Amazon Web Services): Plataforma de servicios de computación en la nube donde se alojará la infraestructura del proyecto.
- Backend: La lógica interna del servidor y manejo de datos que no es visible para el usuario.
- BCrypt: Algoritmo de *hashing* avanzado que añade seguridad extra contra ataques de fuerza bruta mediante un "salado" de datos.
- Build Artifact: El paquete final y ejecutable (como la imagen de Docker) generado tras compilar el código.
- CI/CD (Continuous Integration / Continuous Deployment): Práctica de automatización para la integración y despliegue continuo del código.
- Frontend: Capa de presentación o interfaz gráfica con la que interactúa el usuario.
- Hashing: Proceso irreversible que transforma una contraseña en una cadena de caracteres única para su almacenamiento seguro.
- HTTP/HTTPS: Protocolo de transferencia de hipertexto (seguro) utilizado para las peticiones web.
- JSON (JavaScript Object Notation): Formato ligero de intercambio de datos utilizado entre el cliente y los servicios.
- JWT (JSON Web Token): Estándar para crear tokens de acceso que permiten la autenticación segura.
- Microservicios: Estilo arquitectónico que estructura una aplicación como un conjunto de servicios pequeños e independientes que se comunican entre sí.
- Node.js: Entorno de ejecución para JavaScript en el servidor, ideal para construir microservicios rápidos y escalables.

- PostgreSQL: Sistema de gestión de bases de datos relacional utilizado para almacenar la información del proyecto.
- PWA: Aplicación web con capacidades de app nativa, permitiendo que Hipstagram sea instalable y funcione en móviles y PC.
- RBAC: Control de acceso que asigna permisos según el rol del usuario (ej. un Admin puede borrar posts, un User no).
- Refresh Token: Credencial de larga duración que sirve para renovar el *Access Token* sin que el usuario reingrese sus datos.
- Rol: Conjunto de permisos asignados a un usuario (USER o ADMIN) que define qué acciones puede realizar.
- S3 (Simple Storage Service): Servicio de almacenamiento de objetos de AWS donde se alojarán las imágenes.
- Secure Request: Solicitud de datos cifrada bajo el protocolo HTTPS, que garantiza que la información viaje protegida entre el navegador del usuario y los servidores de AWS.
- Stateless: Modelo donde el servidor no guarda datos de sesión; cada petición debe ser independiente y autocontenida.
- Token: Identificador digital que sirve como "llave" para acceder a recursos protegidos sin exponer la contraseña real.
- User: Rol de cliente final que utiliza las funciones básicas de la red social
- Webhook: Notificación automática de un sistema a otro (ej. de GitHub a AWS) cuando ocurre un evento como un *push* de código.

3 Diagrama de componentes

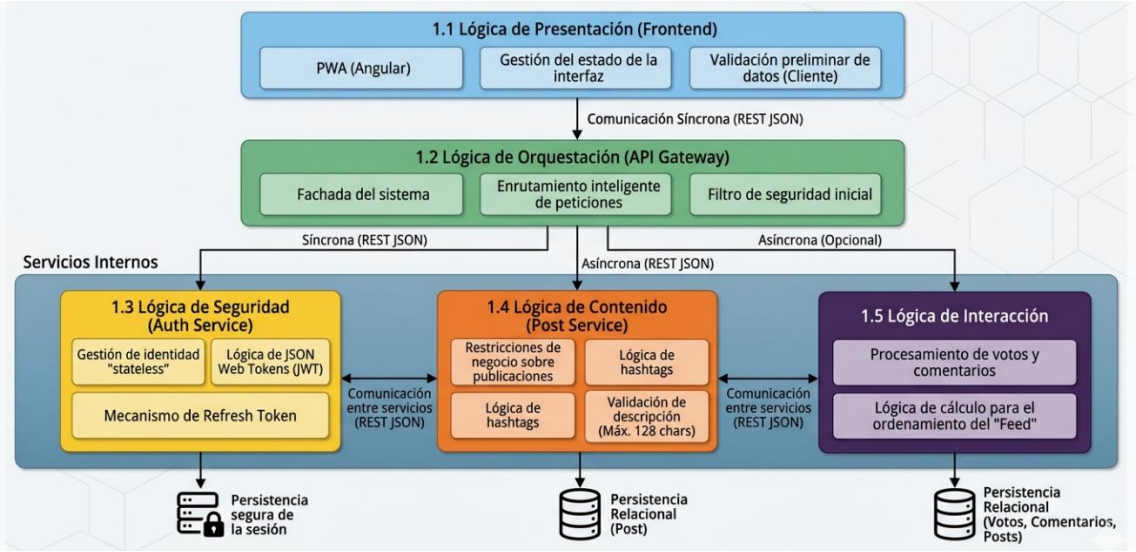


4 Descripción de componentes

El diagrama detalla la infraestructura tecnológica y la interacción entre las diferentes capas del sistema:

- Capa de Cliente (Frontend/ADMIN): Punto de interacción inicial donde se generan las peticiones (HTTP/Secure Request) hacia el servidor.
- API Gateway: Actúa como el orquestador central que recibe todas las solicitudes externas y las redirige al microservicio correspondiente.
- Capa de Lógica de Negocio (Microservicios):
    - Auth-Service: Gestiona la seguridad y validación de identidades.
    - User/Vote-Service: Administra los perfiles de usuario y el registro de interacciones (likes/dislikes).
    - Post/Comment-Service: Controla el ciclo de vida de las publicaciones y los hilos de comentarios.
    - Media/Search-Service: Procesa el contenido multimedia y ejecuta los algoritmos de búsqueda por texto o hashtags.
- Capa de Persistencia (Data Access):
    - PostgreSQL: Almacena la información estructurada y relacional del sistema.
    - AWS (S3): Proporciona almacenamiento en la nube para el contenido multimedia de forma escalable.

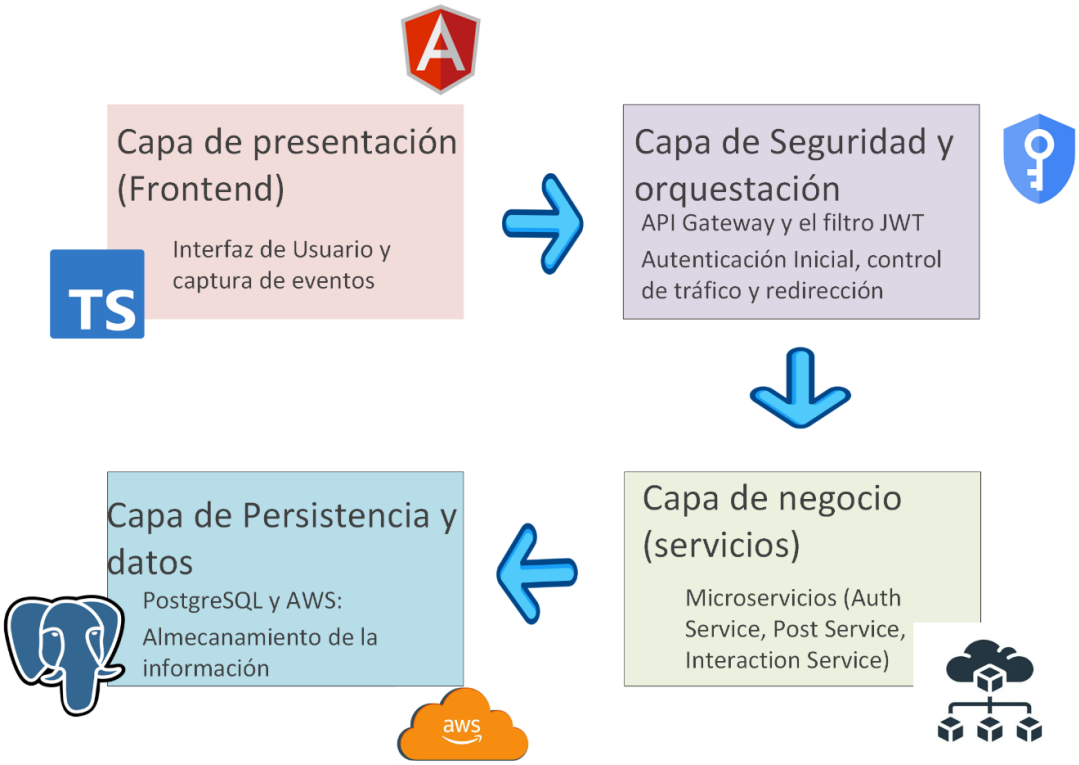
5 Vista lógica que incluya la arquitectura de los componentes de software



Esta arquitectura se basa en la descomposición funcional, donde cada nivel opera de forma autónoma pero integrada mediante contratos definidos:

- Frontend (Angular): Actúa como la capa de experiencia de usuario, gestionando la interacción y validaciones rápidas.
- Gateway: Centraliza el tráfico, garantizando que solo las peticiones seguras y bien enrutadas lleguen al núcleo.
- Servicios Core (Auth, Post, Interacción): Son el motor del sistema. Procesan la seguridad (JWT), aplican reglas de negocio (como el límite de 128 caracteres) y ejecutan algoritmos de ordenamiento para el contenido.
- Persistencia: Cada servicio garantiza la integridad de sus datos en un modelo relacional, permitiendo un escalado independiente y un mantenimiento sin interrupciones globales.

6 Diagrama de capas



El sistema se organiza en un modelo de cuatro capas para asegurar la separación de responsabilidades:

- Presentación: Gestionada por Angular/TS, encargada de la interacción directa con el usuario final.
- Seguridad y Orquestación: Valida los tokens de acceso antes de permitir el paso a los servicios internos.
- Negocio: Donde residen los microservicios que ejecutan las reglas del proyecto Hipstagram.
- Datos: El nivel base donde PostgreSQL garantiza que la información se almacene de forma íntegra y segura.

7 Descripción de la seguridad y credenciales

La seguridad de Hipstagram se implementa bajo un esquema de defensa en capas, asegurando así la integridad, disponibilidad y confidencialidad de los datos.

7.1 Autenticación y Gestión de Sesiones (JWT)

El sistema utiliza JSON Web Tokens (JWT) para el manejo de sesiones stateless. Al iniciar sesión, el servicio de autenticación entrega dos tokens al cliente:

- Access Token: Token de vida corta (ej. 15 min) que viaja en el encabezado de cada petición para autorizar acciones.
- Refresh Token: Token de vida larga almacenado de forma segura que permite obtener nuevos Access Tokens sin que el usuario reingrese sus credenciales.

7.2 Protección de credenciales

Las contraseñas de los usuarios no se almacenan en texto plano. Se utiliza el algoritmo de hashing BCrypt con un factor de costo ajustado, garantizando que ante una eventual fuga de datos en la base de datos PostgreSQL, las credenciales permanezcan cifradas e ilegibles.

7.3 Control de Acceso Basado en Roles (RBAC)

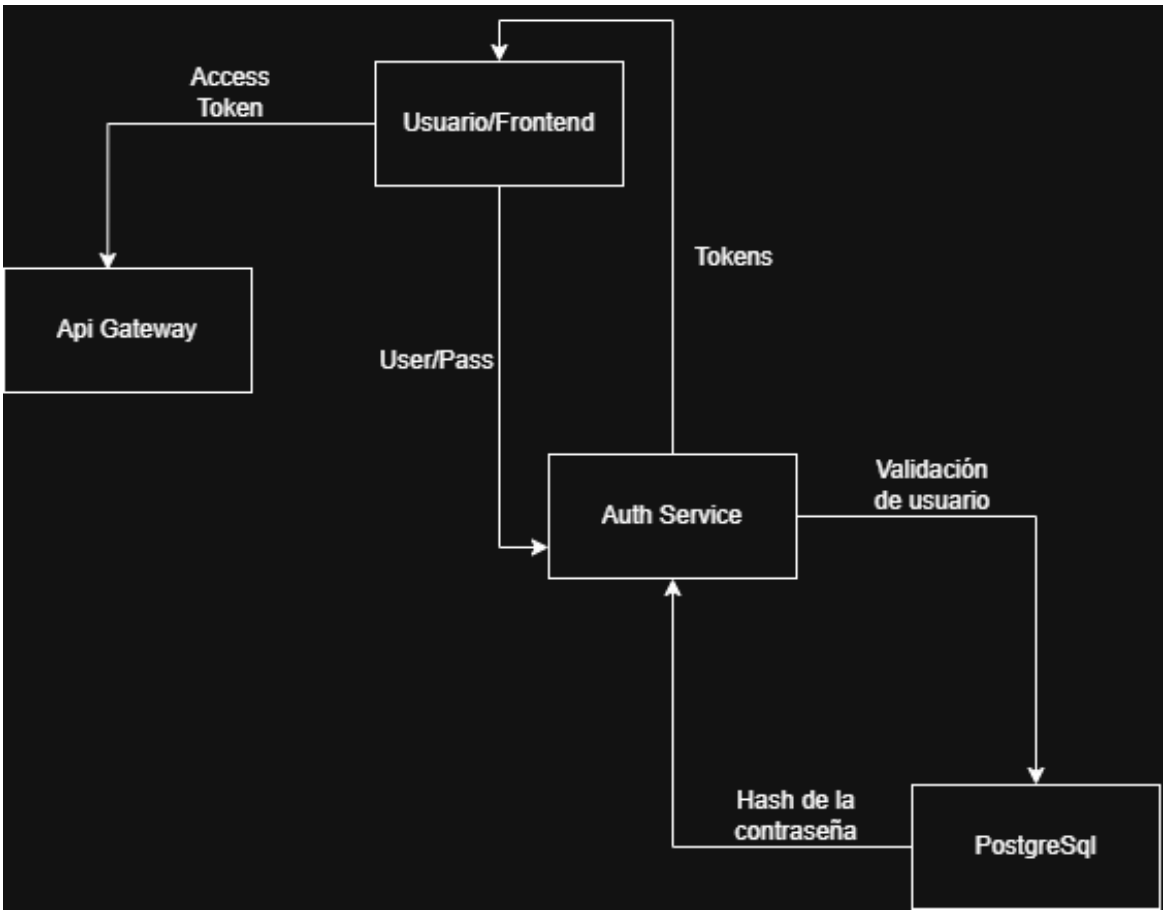
Se definen dos niveles de autoridad para restringir el acceso a los recursos:

- Rol User: Acceso a funcionalidades estándar (publicar, comentar, votar). Solo tiene permisos de edición/borrado sobre sus propios recursos.
- Rol Admin: Posee privilegios elevados para gestionar la plataforma, incluyendo el borrado de cualquier contenido inapropiado y el acceso exclusivo a los Logs de Auditoría.

7.4 Seguridad de la comunicación:

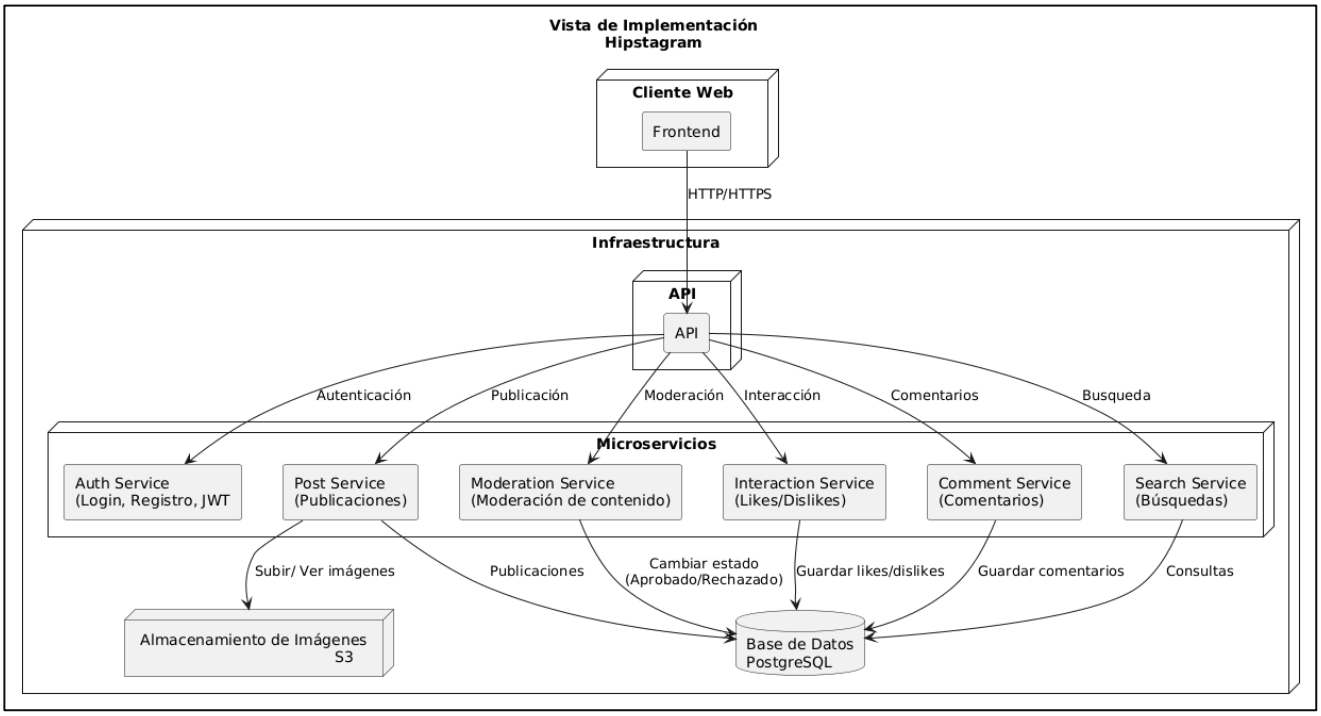
Todo el trafico de datos entre el frontend y el backend se realiza bajo los protocolos HTTPS.

8 Diagrama de seguridad



Este diagrama de seguridad detalla el proceso de autenticación y autorización de Hipstagram, el usuario envía sus credenciales al Auth-Service, el cual valida los datos consultando el hash de la contraseña en PostgreSQL. Una vez validado, el sistema genera Tokens (Access y Refresh) que el frontend almacena. Para cualquier acción posterior, el Api Gateway valida el Access Token antes de permitir la comunicación con los microservicios.

9 Vista de Implementación



El diagrama ilustra la estructura de tres capas:

- Frontend: Un cliente web que se comunica mediante HTTP/HTTPS.

- Infraestructura: Un API Gateway central que distribuye las peticiones hacia seis microservicios especializados (Autenticación, Publicaciones, Moderación, Interacciones, Comentarios y Búsquedas).

- Persistencia: El almacenamiento de archivos multimedia se gestiona en Amazon S3, mientras que la lógica de datos relacionales se centraliza en una base de datos PostgreSQL.

10 Descripción general de la implementación

Para garantizar la portabilidad y escalabilidad de Hipstagram, se ha seleccionado una arquitectura basada en Docker. Cada componente de la aplicación (Frontend en Angular, microservicios en Node.js y la base de datos PostgreSQL) se empaqueta en contenedores independientes.

El proceso de implementación sigue un flujo de integración y despliegue continuo (CI/CD):

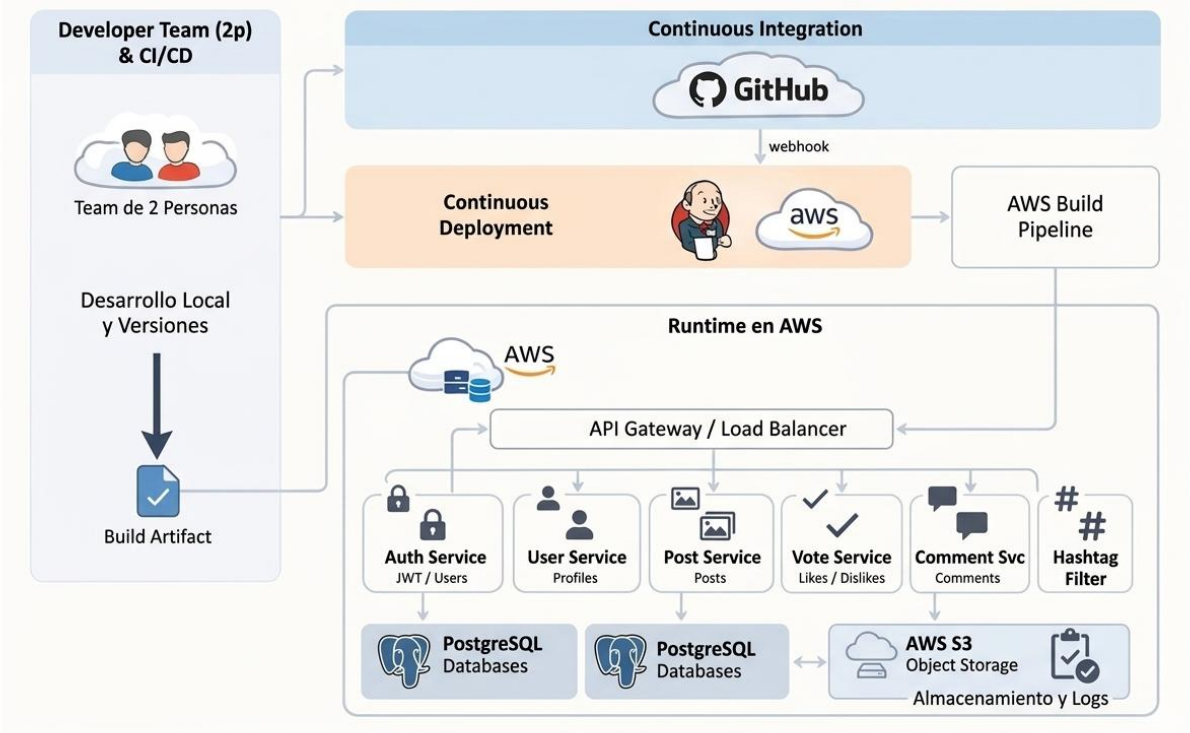
1. Desarrollo y Control de Versiones: El equipo de 2 personas trabaja en un entorno local y sube el código a GitHub.

2. Automatización (Pipeline): Mediante un Webhook, se activa un proceso automático que construye las imágenes de Docker.

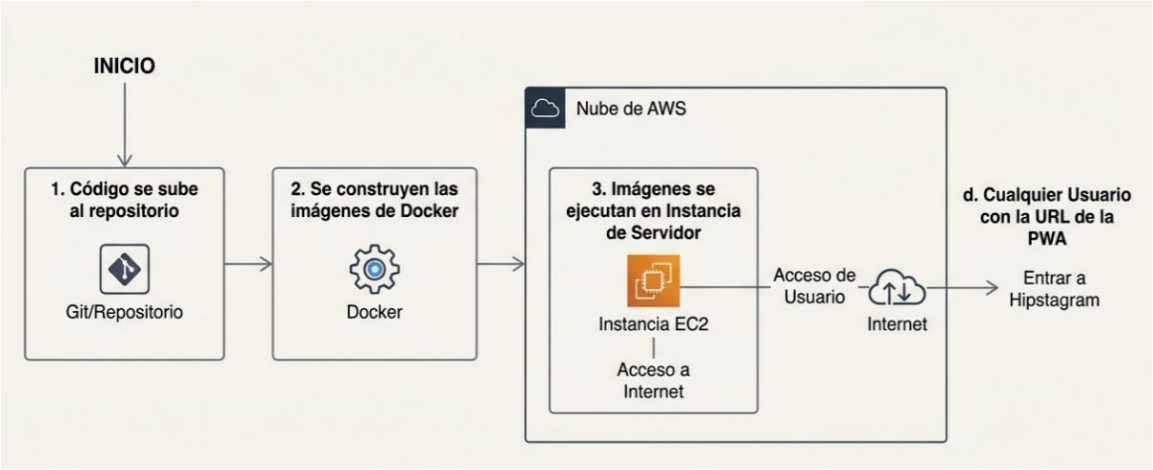
3. Despliegue en la Nube (AWS): Estas imágenes se despliegan en la infraestructura de AWS, donde un API Gateway distribuye el tráfico hacia los distintos microservicios.

4. Persistencia y Almacenamiento: Se utilizan instancias de PostgreSQL para los datos relacionales y AWS S3 para el almacenamiento de archivos multimedia y logs, asegurando que toda la infraestructura esté conectada y lista para el usuario final desde el primer segundo.

11 Diagrama de la implementación



12 Vista de Puesta en Marcha



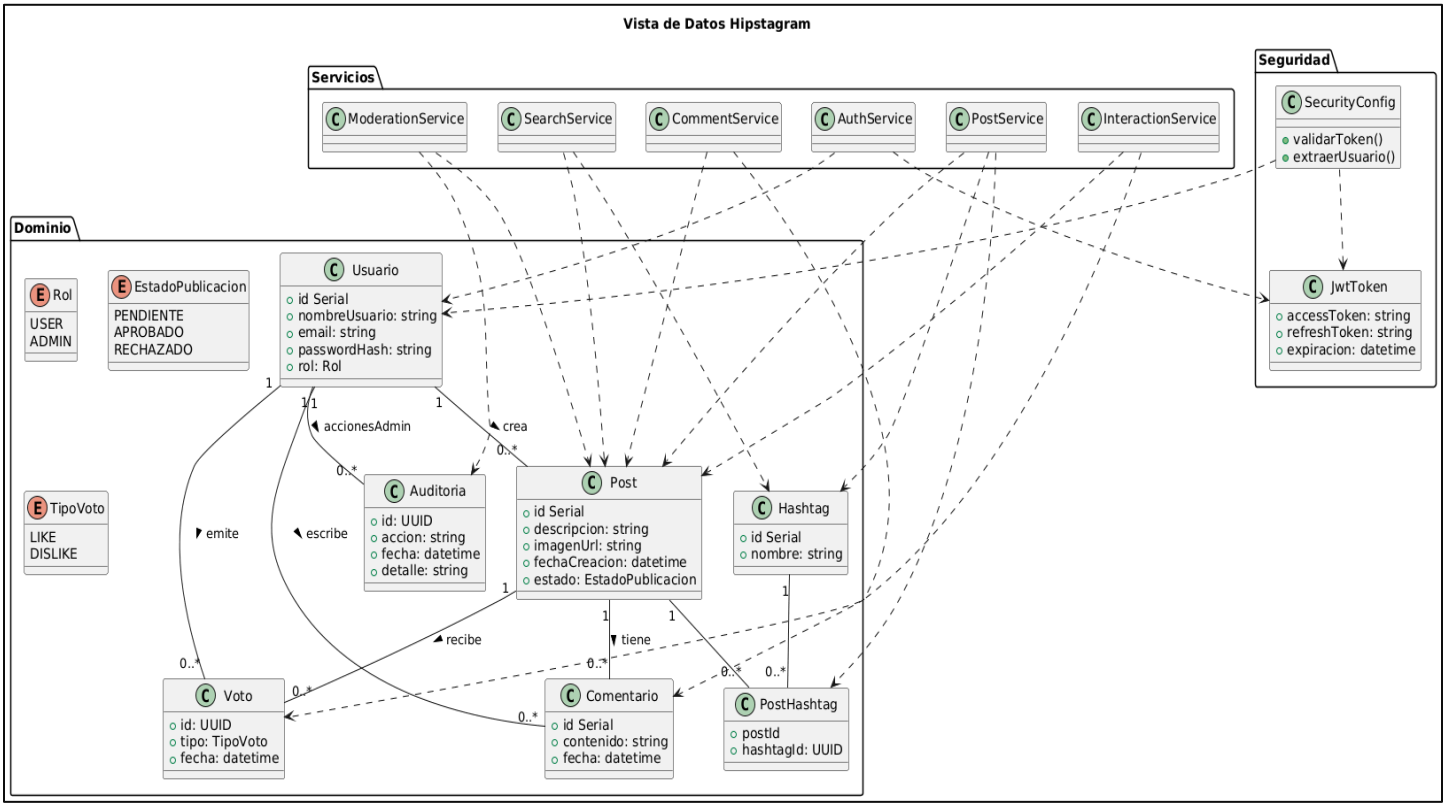
La aplicación se desplegará utilizando una infraestructura de nube (AWS). El flujo será el siguiente:

1. El código se sube a un repositorio.

2. Se construyen las imágenes de Docker.

3. Estas imágenes se ejecutan en una instancia de servidor que tiene acceso a internet, permitiendo que cualquier usuario con la URL de la PWA pueda entrar a Hipstagram.

13 Vista de Datos



Esta Vista de Datos organiza la estructura lógica del proyecto en tres capas:

13.1 Capa de Dominio (Entidades de Negocio)

Es el núcleo del sistema y define cómo se relacionan los datos principales:

- Usuarios y Roles: Maneja la información de perfil y distingue entre usuarios normales y administradores.
- Publicaciones (Post): El elemento central, que incluye imágenes, estados de aprobación (Pendiente, Aprobado, Rechazado) y su relación con etiquetas (Hashtag).
- Interacciones: Registra los Comentarios y los Votos (Like/Dislike) que los usuarios realizan sobre los posts.
- Auditoría: Un componente para rastrear acciones administrativas y mantener un historial de cambios.

13.2 Capa de Servicios

Define la lógica que manipula los datos. Cada servicio tiene una responsabilidad única (Microservicios):

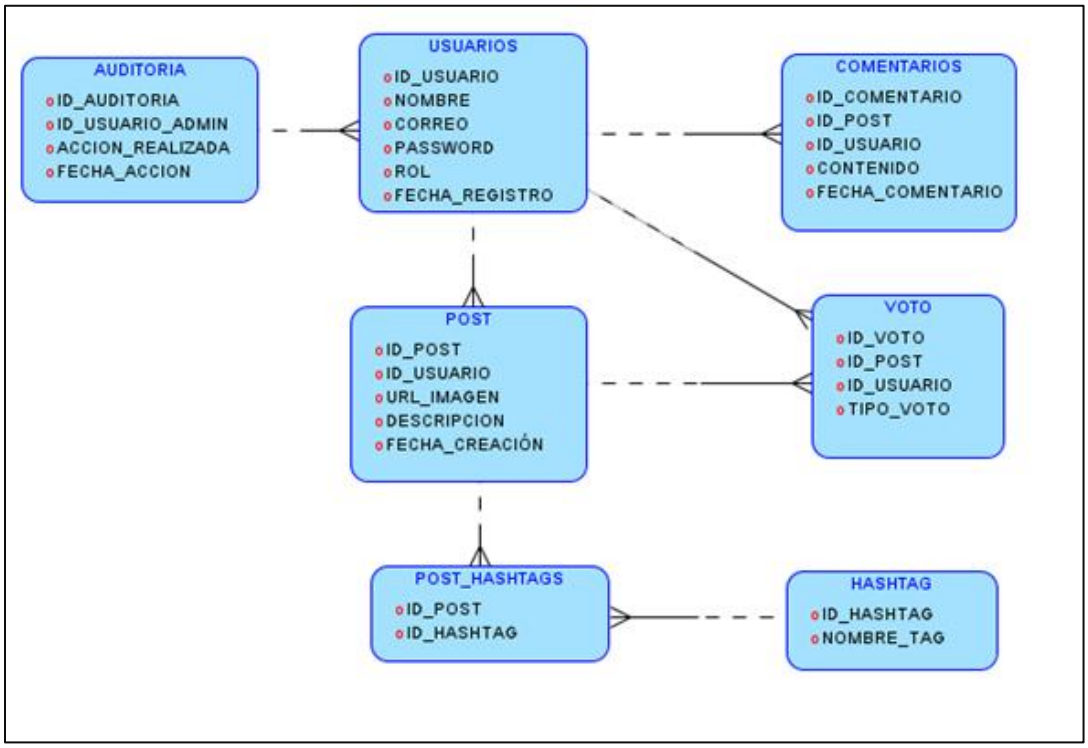
- AuthService: Gestión de usuarios y sesiones.
- PostService: Creación y manejo de publicaciones.
- Comment e InteractionService: Gestión de la actividad social.
- Moderation y SearchService: Filtro de contenido y búsqueda.

13.3 Capa de Seguridad

Se enfoca en la protección de los datos y el acceso:

- SecurityConfig: Contiene la lógica para validar identidades.
- JwtToken: Utiliza tokens (Access y Refresh) para que las comunicaciones entre el frontend y los microservicios sean seguras y autenticadas.

14 Diagrama Entidad-Relación (DER)



En este Modelo Entidad-Relación (ER) se detallan las tablas, sus atributos principales y cómo se conectan entre sí para almacenar la información en PostgreSQL.

Puntos clave:

- Núcleo de Usuarios: La tabla USUARIOS es el centro de la aplicación, conectándose con todas las interacciones sociales (votos, comentarios y publicaciones).
- Gestión de Contenido: La tabla POST almacena la información de las imágenes (URL) y se vincula con POST_HASHTAGS y HASHTAG mediante una relación de muchos a muchos, permitiendo categorizar las fotos.
- Interacciones Sociales: Las tablas COMENTARIOS y VOTO registran la actividad de los usuarios sobre los posts.
- Control Administrativo: La tabla AUDITORIA registra las acciones realizadas por los administradores (basado en el campo ROL de la tabla de usuarios), lo cual es fundamental para la seguridad y el seguimiento del sistema.

15 Diccionario de Datos

Especificaciones de las entidades principales.

Usuario			
Campo	Tipo	Atributos	Descripción
id_usuario	INT	PK, AI	Identificador único del usuario.
nombre	VARCHAR(50)	NOT NULL	Nombre de usuario o nick.
correo	VARCHAR(100)	UNIQUE	Correo electrónico para el login.
contraseña	VARCHAR(255)	NOT NULL	Contraseña (hash).
rol	VARCHAR(20)	DEFAULT 'User'	Define si es Usuario o Admin
Fecha_registro	DATETIME	DEFAULT NOW	Fecha de inicio de sesión

Auditoría			
Campo	Tipo	Atributos	Descripción
id_auditoria	INT	PK, AI	ID del registro de auditoría.
id_usuario_admin	INT	FK	Referencia al admin que actuó.
Acción_realizada	VARCHAR(100)	NOT NULL	Descripción de la acción realizada.
Fecha_accion	DATETIME	DEFAULT NOW	Momento exacto de la acción.

Post			
Campo	Tipo	Atributos	Descripción
id_post	INT	PK, AI	Identificador único de la publicación.
id_usuario	INT	FK	ID del autor (viene del servicio Auth).
url_imagen	VARCHAR(255)	NOT NULL	Ruta o enlace de la imagen.
descripcion	VARCHAR(128)		Pie de foto (máx. 128 caracteres).
Fecha_creacion	DATETIME	DEFAULT NOW	Fecha de creación del post.

Votos			
Campo	Tipo	Atributos	Descripción
id_voto	INT	PK, AI	Identificador único del voto.
id_post	INT	FK	Publicación que recibe el voto.
id_usuario	INT	FK	Usuario que emite el voto.
Tipo_voto	INT	CHECK (-1, 1)	1 para Like, -1 para Dislike.

Hashtag			
Campo	Tipo	Atributos	Descripción
id_hashtag	INT	PK, AI	Identificador de la etiqueta.
nombre_tag	VARCHAR(50)	UNIQUE	El texto del hashtag (ej: #tecnologia).

Comentarios			
Campo	Tipo	Atributos	Descripción
id_comentario	INT	PK, AI	Identificador único del comentario.
id_post	INT	FK	Referencia al post donde se comenta.
id_usuario	INT	FK	ID del usuario que escribe (desde Auth).
contenido	TEXT	NOT NULL	El texto del comentario (sin límite corto).
fecha_comentario	DATETIME	DEFAULT NOW	Momento en que se publicó.

Post_hashtag			
Campo	Tipo	Atributos	Descripción
id_post	INT	PK, AI	Relación con la tabla Post
Id_hashtag	INT	PK	Relación con la tabla Hashtag

16 Definición de Roles de la Aplicación

Para mantener el orden, se definieron dos roles principales:

1. Usuario (USER): Puede registrarse, subir fotos, comentar y votar (like/dislike). Solo puede editar o borrar su propio contenido.

2. Administrador (ADMIN): Posee un rol con privilegios. Puede ver la tabla de auditoría, eliminar cualquier post o comentario que rompa las reglas y gestionar a otros usuarios.

17 Alcance y Limitaciones

17.1 Alcance

- Registro de nuevos usuarios por medio de credenciales (Correo y contraseña)
- Implementación de tokens de seguridad para inicio de sesiones (JWT)
- Soporte y gestión de tipos de usuario(User/Admin)
- Publicación de imágenes con descripciones y hashtags
- Moderación de contenido manual por medio del rol Admin
- Sistema de votaciones por likes/dislikes
- Sistema de comentarios
- Búsqueda de publicaciones por texto libre, hashtags y usuario
- Feed ordenado por número de likes
- Panel administrativo exclusivo del rol Admin
- Despliegue en nube AWS
- Contenerización en contenedores (Docker)
- Repositorio de versiones en GitHub
- Microservicios independientes
- Base de datos relacional en PostgreSQL
- Almacenamiento de imágenes en nube (S3)
- Moderación de contenido por texto e imágenes automático

17.2 Limitaciones

- Sistema de mensajería entre usuarios
- Sistema de notificaciones
- Sistema de integración de vídeos de ningún tipo
- Publicidad
- Streaming
- Solicitudes de amistad entre usuarios
- Cuentas empresariales, páginas o grupos
- Sistema de compraventa (Marketplace)
- Publicaciones con más de 128 caracteres
- Integración con plataformas externas
- Sistema de seguridad facial o dactilar
- Sistema de historias o publicaciones temporales
- Sistema de cuentas múltiples.
- Ninguna integración con servicios de IA
- Reacciones diferentes de like/Dislike
- Transacciones de ningún tipo.
- Ningún tipo de suscripción o membresía.